COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010/Data Structure Algorithm | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | July 23, 2026 | **DATE SUBMITTED:** | July 28, 2026 |
| **NAME:** | Jaime D.C. Godino Jr. | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**ACTIVITY NO. 3.1** **Linked List**

## 1. PROCEDURE OUTPUT

**ILO A: Construct C++ code for a singly and doubly linked list in C++**

*Table 3-1. Output of Initial/Simple Implementation*

**CODE**

```
#include <iostream>
#include "singly_LL.h"

int main(){

    // creating an empty list
    SinglyList<char>* head = new SinglyList<char>;
    SinglyList<char>* second = new SinglyList<char>;
    SinglyList<char>* third = new SinglyList<char>;
    SinglyList<char>* fourth = new SinglyList<char>;
    SinglyList<char>* fifth = new SinglyList<char>;

    //assign the data:
    head->data = 'J';
    head->next = second;

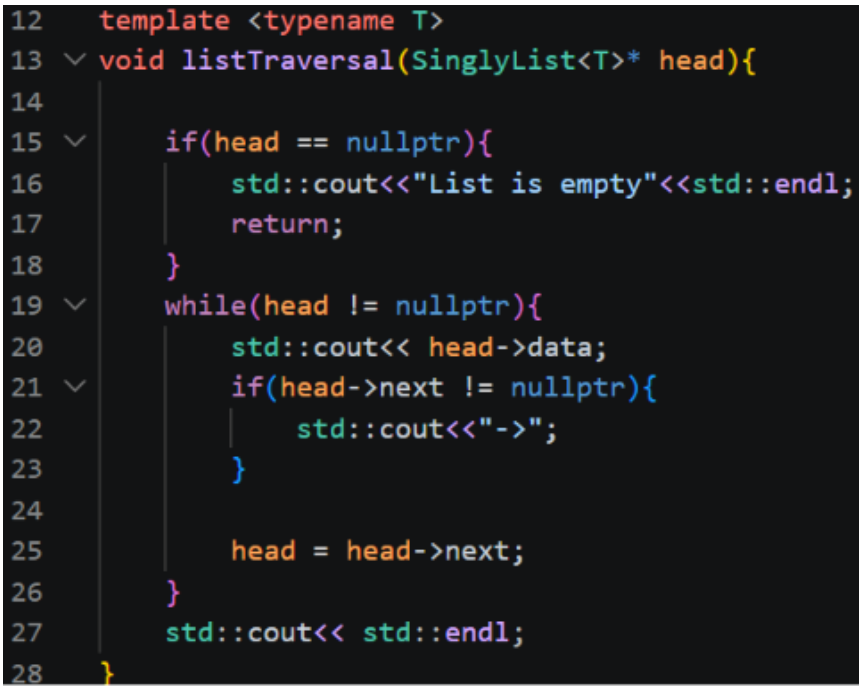
    second->data = 'A';
    second->next = third;

    third->data = 'I';
    third->next = fourth;

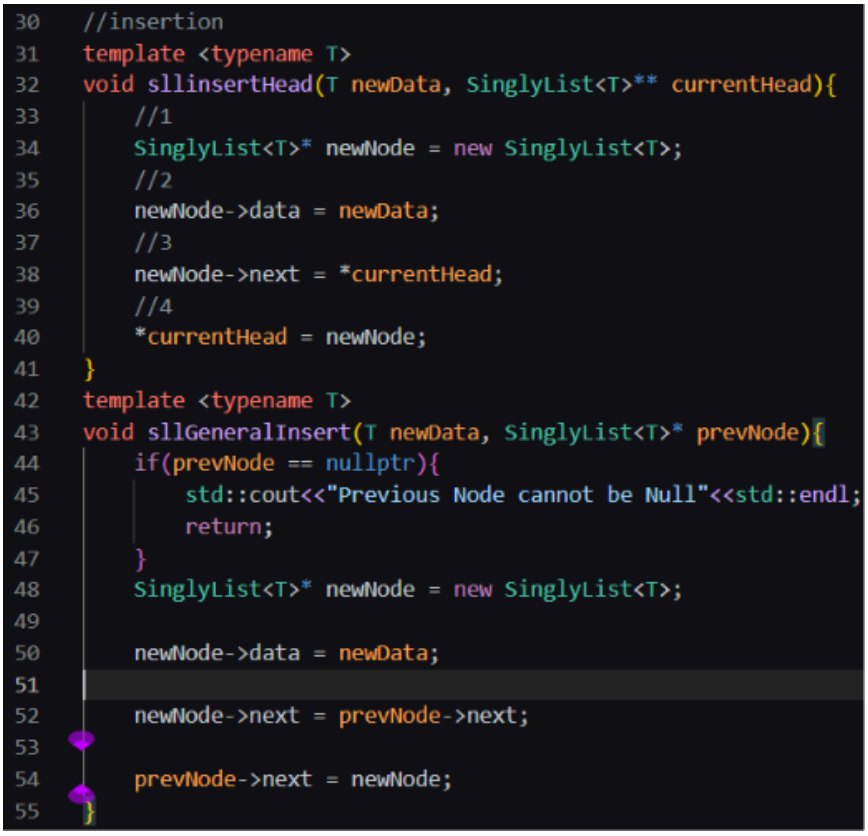
    fourth->data = 'G';
    fourth->next = nullptr;
```

| OBSERVATION |
| --- |
| The program created and linked each node to the next using the next pointer. Each node is assigned a character, which forms a sequence J->A->I->G. The program could use operations such as traversal, insert node, and delete node so that there will be an output displayed. |

*Table 3-2. Code for the List Operations*

| OPERATION | SCREENSHOT |
| --- | --- |
| Traversal |  |

```
12  template <typename T>
13  void listTraversal(SinglyList<T>* head){
14
15      if(head == nullptr){
16          std::cout<<"List is empty"<<std::endl;
17          return;
18      }
19      while(head != nullptr){
20          std::cout<< head->data;
21          if(head->next != nullptr){
22              std::cout<<"->";
23          }
24
25          head = head->next;
26      }
27      std::cout<< std::endl;
28  }
```

| | |
|---|---|
| Insertion at head |  |
| Insertion at any part of the list |  |

```cpp
//insertion
template <typename T>
void sllinsertHead(T newData, SinglyList<T>** currentHead){
    //1
    SinglyList<T>* newNode = new SinglyList<T>;
    //2
    newNode->data = newData;
    //3
    newNode->next = *currentHead;
    //4
    *currentHead = newNode;
}
template <typename T>
void sllGeneralInsert(T newData, SinglyList<T>* prevNode){
    if(prevNode == nullptr){
        std::cout<<"Previous Node cannot be Null"<<std::endl;
        return;
    }
    SinglyList<T>* newNode = new SinglyList<T>;

    newNode->data = newData;

    newNode->next = prevNode->next;

    prevNode->next = newNode;
}
```

```cpp
template <typename T>
void sllGeneralInsert(T newData, SinglyList<T>* prevNode){
    if(prevNode == nullptr){
        std::cout<<"Previous Node cannot be Null"<<std::endl;
        return;
    }
    SinglyList<T>* newNode = new SinglyList<T>;

    newNode->data = newData;

    newNode->next = prevNode->next;

    prevNode->next = newNode;
}
```

Insertion at the end

```
57  template <typename T>
58  void sllinsertEnd(T newData, SinglyList<T>** head){
59
60      SinglyList<T>* newNode= new SinglyList<T>;
61
62      SinglyList<T>* currentNode = *head;
63
64      newNode->data = newData;
65
66      newNode->next = nullptr;
67
68      while(currentNode->next !=nullptr){
69          currentNode = currentNode->next;
70      }
71
72      currentNode->next = newNode;
73  }
74
```

Deletion of a node

```
75  template <typename T>
76  void sllDeleteNode(T findData, SinglyList<T>** head){
77      //check
78      if(*head == nullptr) return;
79
80      SinglyList<T>* currNode = *head;
81      SinglyList<T>* prevNode = nullptr;
82
83      //search loop
84      while(currNode != nullptr && currNode->data != findData){
85          prevNode = currNode;
86          currNode = currNode->next;
87      }
88
89      //data was found:
90      if(currNode == nullptr) return;
91
92      //what if
93      if(prevNode ==nullptr){
94          *head = currNode->next;
95      }else{
96          prevNode->next = currNode->next;
97      }
98      //delete
99      delete currNode;
100
101 }
102
```

Delete all list

```
103   template <typename T>
104   void sllDeleteList(SinglyList<T>** head){
105       SinglyList<T>* current = *head;
106
107       while(current != nullptr){
108           SinglyList<T>* temp = current;
109           current = current->next;
110           delete temp;
111       }
112       *head = nullptr;
113   }
114
115   #endif
```

a. Traverse the list by passing the head of the created list into the function

MAIN.CPP

```
39      std::cout<<"original list: \n";
40      listTraversal(head);
```

SINGLY.H

```
12  template <typename T>
13  void listTraversal(SinglyList<T>* head){
14
15      if(head == nullptr){
16          std::cout<<"List is empty"<<std::endl;
17          return;
18      }
19      while(head != nullptr){
20          std::cout<< head->data;
21          if(head->next != nullptr){
22              std::cout<<"->";
23          }
24
25          head = head->next;
26      }
27      std::cout<< std::endl;
28  }
```

OUTPUT

```
original list:
C->P->E->0->1->0
```

**b. Insert the element 'G' at the start of the list to replace the current node. Output should now show "GCPE101"**

**MAIN.CPP**

```
std::cout<<"inserting a new node at the head: \n";
sllinsertHead('Q', &head);
listTraversal(head);
```

**SINGLY.H**

```
30  //insertion
31  template <typename T>
32  void sllinsertHead(T newData, SinglyList<T>** currentHead){
33      //1
34      SinglyList<T>* newNode = new SinglyList<T>;
35      //2
36      newNode->data = newData;
37      //3
38      newNode->next = *currentHead;
39      //4
40      *currentHead = newNode;
41  }
42  template <typename T>
43  void sllGeneralInsert(T newData, SinglyList<T>* prevNode){
44      if(prevNode == nullptr){
45          std::cout<<"Previous Node cannot be Null"<<std::endl;
46          return;
47      }
48      SinglyList<T>* newNode = new SinglyList<T>;
49
50      newNode->data = newData;
51
52      newNode->next = prevNode->next;
53
54      prevNode->next = newNode;
55  }
```

**OUTPUT**

```
inserting a new node at the head:
G->C->P->E->0->1->0
```

**c. Insert an element "E" with the previous node element being "P". Output should now show "GCPEE101".**

**MAIN.CPP**

```
41      std::cout<<"inserting a new node at the third: \n";
42      sllGeneralInsert('E', head->next->next);
```

**SINGLY.H**

```
42  template <typename T>
43  void sllGeneralInsert(T newData, SinglyList<T>* prevNode){
44      if(prevNode == nullptr){
45          std::cout<<"Previous Node cannot be Null"<<std::endl;
46          return;
47      }
48      SinglyList<T>* newNode = new SinglyList<T>;
49
50      newNode->data = newData;
51
52      newNode->next = prevNode->next;
53
54      prevNode->next = newNode;
55  }
56
```

**OUTPUT**

```
inserting a new node at the third:
G->C->P->E->E->0->1->0
```

d. Delete the node containing the element C.

**MAIN.CPP**

```
48      std::cout<<"deleting a node: \n";
49      sllDeleteNode('C', &head);
50      listTraversal(head);
```

**SINGLY.H**

```cpp
template <typename T>
void sllDeleteNode(T findData, SinglyList<T>** head){
    //check
    if(*head == nullptr) return;

    SinglyList<T>* currNode = *head;
    SinglyList<T>* prevNode = nullptr;

    //search loop
    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }

    //data was found:
    if(currNode == nullptr) return;

    //what if
    if(prevNode ==nullptr){
        *head = currNode->next;
    }else{
        prevNode->next = currNode->next;
    }
    //delete
    delete currNode;

}
```

**OUTPUT**

```
deleting a node:
G->P->E->0->1->0
```

**e. Delete the node containing the element P.**

**MAIN.CPP**

```cpp
    std::cout<<"deleting a node: \n";
    sllDeleteNode('P', &head);
    listTraversal(head);
```

**SINGLY.H**

```cpp
template <typename T>
void sllDeleteNode(T findData, SinglyList<T>** head){
    //check
    if(*head == nullptr) return;

    SinglyList<T>* currNode = *head;
    SinglyList<T>* prevNode = nullptr;

    //search loop
    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }

    //data was found:
    if(currNode == nullptr) return;

    //what if
    if(prevNode ==nullptr){
        *head = currNode->next;
    }else{
        prevNode->next = currNode->next;
    }
    //delete
    delete currNode;

}
```

**OUTPUT**

```
deleting a node:
G->E->E->0->1->0
```

f. Show the elements in the list. Output should be “GEE101”

**MAIN.CPP**

```cpp
    std::cout<<"final list: \n";
    listTraversal(head);
```

**OUTPUT**

```
deleting a node:
G->E->E->0->1->0
```

*Table 3-4. Modified Operations for Doubly Linked Lists*

**1. MODIFICATION**

```
4   //TEMPLATING
5   template <typename T>
6   class DoubleList{
7       public:
8           T data;
9           DoubleList<T> *next = nullptr;
10          DoubleList<T> *prev = nullptr;
11
12  };
```

**ANALYSIS**

This visits each node one by one, starting from the head and following the next pointers until the end of the list is reached. This operation checks that all nodes are properly connected and displays the stored data in the correct order.

**2. MODIFICATION**

```
64  template <typename T>
65  void dllInsertHead(T newData, DoubleList<T>** currentHead){
66      //create a node
67      DoubleList<T>* newNode = createNode(newData);
68      //new node should point to the current head
69      newNode->next = *currentHead;
70
71      //current head should point back to the new
72      (*currentHead)->prev = newNode;
73
74      //update header pointer
75      *currentHead = newNode;
76  }
```

**ANALYSIS**

This is responsible for inserting the new node before the current head, and the head pinter is updated to point at the new node. This makes the inserted element the first node in the list while maintaining the connection of the remaining nodes.

**3. MODIFICATION**

```
78  template <typename T>
79  void dllInsertEnd(T newData, DoubleList<T> *currentHead){
80
81      //create a new node
82      DoubleList<T> *newNode = createNode(newData);
83      //traverse to the last node
84      while(currentHead->next !=nullptr){
85          currentHead = currentHead->next;
86      }
87      //connect the last node to the new node
88      currentHead->next = newNode;
89      //connect the new node back to the last node
90      newNode->prev = currentHead;
91  }
```

**ANALYSIS**

This traverses the list until it reaches the last node. The last node's next pointer is then updated to reference the new node, which allows the new element to be appended without affecting the existing order.

**4. MODIFICATION**

```
93  template <typename T>
94  void dllGeneralInsert(T newData, DoubleList<T>* currentNode) {
95      DoubleList<T>* newNode = createNode(newData);
96
97      newNode->next = currentNode->next;
98      newNode->prev = currentNode;
99
100     currentNode->next = newNode;
101
102     if (newNode->next != nullptr) {
103         newNode->next->prev = newNode;
104     }
105 }
```

**ANALYSIS**

This searches for the specified previous node before inserting the new node. The new node's pointer is linked to the next node, and the previous node is updated to point to the new node.

**5. MODIFICATION**

```cpp
template <typename T>
void dllDeleteNode(T findData, DoubleList<T>** head) {

    if (*head == nullptr) {
        std::cout << "List is empty" << std::endl;
        return;
    }

    DoubleList<T>* currentNode = *head;

    while (currentNode != nullptr && currentNode->data != findData) {
        currentNode = currentNode->next;
    }

    if (currentNode == nullptr) {
        std::cout << "Data not found" << std::endl;
        return;
    }

    if (currentNode->prev != nullptr) {
        currentNode->prev->next = currentNode->next;
    } else {
        *head = currentNode->next;
    }

    if (currentNode->next != nullptr) {
        currentNode->next->prev = currentNode->prev;
    }

    delete currentNode;
}
```

**ANALYSIS**

This locates the target node and reconnects the other nodes so that the deleted node is removed from the list. The node's memory is then deallocated to prevent memory leaks while keeping the list intact.

## 6. MODIFICATION

```cpp
template <typename T>
void dllDeleteAll(DoubleList<T>** currentHead) {
    DoubleList<T>* temp;

    while (*currentHead != nullptr) {
        temp = *currentHead;
        *currentHead = (*currentHead)->next;
        delete temp;
    }
}
```

**ANALYSIS**

This is responsible for repeatedly deleting the head node until the list becomes empty. This ensures that all dynamically allocated memory is released and leaves the head pointer as nullptr, which indicates an empty list.

**7. MODIFICATION**

```
33      //checking if the list empty
34      if (currentNode == nullptr) {
35          std::cout << "The list is empty";
36          return;
37      }
38
39      //forward traversing:
40      std::cout << "Forward: " << std::endl;
41      //move towards the last node (till it becomes nullptr)
42      while (currentNode != nullptr) {
43          //print the data
44          std::cout << currentNode->data << " ";
45          //save the current node as the last visited node
46          tail = currentNode;
47          //move to the next node
48          currentNode = currentNode->next;
49      }
50      std::cout << std::endl;
51
52      //backward traversing:
53      std::cout << "Backward: " << std::endl;
54      //move toward the head
55      while (tail != nullptr) {
56          //print the data
57          std::cout << tail->data << " ";
58          //move to the previous node
59          tail = tail->prev;
60      }
61      std::cout << std::endl;
62  }
```

**ANALYSIS**

This doubly linked list is responsible for storing both next and prev pointers in each node. This allows forward and backward traversal and simplifies insertion and deletion operations because neighboring nodes can be accessed directly without searching for the previous node.

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**ILO B: Solve given problems utilizing linked lists in C++**

**CODE**

supplementarymain.cpp

```cpp
#include <iostream>
#include <string>
#include "circular_DLL.h"

int main() {

    DoubleList<std::string>* playlist = nullptr;
    DoubleList<std::string>* currTrack = nullptr;
    std::string title;
    int choice = 0;

    do{
        std::cout << "1 - Play All\n";
        std::cout << "2 - Next Song\n";
        std::cout << "3 - Previous Song\n";
        std::cout << "4 - Add Song\n";
        std::cout << "5 - Remove Song\n";
        std::cout << "6 - Delete Playlist\n";

        std::cout << "0 - Exit\n";
        std::cin >> choice;
        std::cin.ignore();

        switch(choice) {
            case 1:
                cdllTraversal(playlist);
                break;

            case 2:
                next(&currTrack);
                break;
```

```
33              case 3:
34                  previous(&currTrack);
35                  break;
36
37              case 4:
38                  std::cout << "Enter song title you want to add"<<std::endl;
39                  std::getline(std::cin, title);
40                  cdllInsertEnd(title, &playlist);
41                  if (currTrack == nullptr) {
42                      currTrack = playlist;
43                  }
44                  std::cout << title <<" has been added"<<std::endl;
45                  break;
46
47              case 5:
48                  if (playlist == nullptr) {
49                      std::cout << "EMPTY PLAYLIST"<<std::endl;
50                      break;
51
52                  }
53                  std::cout << "Enter song title to remove: ";
54                  std::getline(std::cin, title);
55                  cdllDeleteNode(title, &playlist);
56                  std::cout << title <<" has been removed"<<std::endl;
57
58                  if (playlist == nullptr) {
59                      currTrack = nullptr;
60                  }
```

```
61                  break;
62
63              case 6:
64                  std::cout << "DELETING PLAYLIST"<<std::endl;
65                  cdllDeleteList(&playlist);
66                  currTrack = nullptr;
67                  break;
68
69              case 0:
70                  std::cout << "Exiting..."<<std::endl;
71                  break;
72
73              default:
74                  std::cout << "INVALID CHOICE"<<std::endl;
75                  break;
76          }
77
78      } while(choice != 0);
79
80
81  return 0;
82
83  }
84
```

circular_LL.h

```h
#ifndef CIRCULAR_DLL_H
#define CIRCULAR_DLL_H

template <typename T>
class DoubleList{
    public:
    T data;
    DoubleList<T>* prev = nullptr;
    DoubleList<T>* next = nullptr;

};

// create a new node;

template <typename T>
DoubleList<T>* createNode(T newData) {
    //dynamically allocate memory
    DoubleList<T>* newNode = new DoubleList<T>;

    newNode->data = newData;
    newNode->prev = nullptr;
    newNode->next = nullptr;
    //return the address
    return newNode;
}
template <typename T>
void cdllTraversal(DoubleList<T>* head) {
    //checking if the list empty
    if (head == nullptr) {
        std::cout<<"Empty Playlist"<<std::endl;
        return;
    }
```

```
33      DoubleList<T>* current = head;
34      do {
35          std::cout<<current->data<<"->";
36          current = current->next;
37
38      }
39      while (current != head);
40      std::cout<<"Playlist Restarted"<<std::endl;
41
42  }
43
44  template <typename T>
45  void cdllInsertEnd(T newData, DoubleList<T>** head) {
46      DoubleList<T>* newNode = createNode(newData);
47      if (*head == nullptr) {
48          *head = newNode;
49          newNode->next = *head;
50          newNode->prev = *head;
51          return;
52
53      }
54      DoubleList<T>* tail = (*head)->prev;
55      newNode->next = *head;
56      newNode->prev = tail;
57      tail->next = newNode;
58      (*head)->prev = newNode;
59
60  }
```

```
61
62  template <typename T>
63  void cdllDeleteNode(T findData, DoubleList<T>** head) {
64      if (*head == nullptr) return;
65      DoubleList<T>* currNode = *head;
66
67      do {
68          if (currNode->data == findData) break;
69          currNode = currNode->next;
70      }
71      while (currNode != *head);
72
73      if (currNode->data != findData) return;
74
75      if (currNode->next == currNode) {
76          delete currNode;
77          *head = nullptr;
78          return;
79      }
80
81      if (currNode == *head) {
82          *head = currNode->next;
83      }
84
85      currNode->prev->next = currNode->next;
86      currNode->next->prev = currNode->prev;
87      delete currNode;
88  }
89
```

```
90  template <typename T>
91  void cdllDeleteList(DoubleList<T>** head) {
92      if (head == nullptr || *head == nullptr) return;
93      DoubleList<T>* current = *head;
94      (*head)->prev->next = nullptr;
95
96      while (current != nullptr) {
97          DoubleList<T>* temp = current;
98          current = current->next;
99          delete temp;
100     }
101     *head = nullptr;
102 }
103
104 template <typename T>
105 void next(DoubleList<T>** currTrack) {
106     if (currTrack == nullptr || *currTrack == nullptr){
107         std::cout<<"Empty Playlist"<<std::endl;
108         return;
109     }
110     *currTrack = (*currTrack)->next;
111     std::cout<<(*currTrack)->data<<" is now playing"<<std::endl;
112 }
113
114 template <typename T>
115 void previous(DoubleList<T>** currTrack) {
116     if (currTrack == nullptr || *currTrack == nullptr) {
117         std::cout<<"Empty Playlist"<<std::endl;
118         return;
119     }
120     *currTrack = (*currTrack)->prev;
121     std::cout<<(*currTrack)->data<<" is now playing"<<std::endl;
122 }
123
124 #endif
```

## OUTPUT

1. Menu

```
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - Delete Playlist
0 - Exit
```

2. Play All

```
Are You Okay?- Daniel Caesar->Who Hurt You?- Daniel Caesar->Do You Like Me?- Daniel Caesar->Playlist Restarted
```

3. Next Song

```
Do You Like Me?- Daniel Caesar is now playing
```

4. Previous Song

```
Who Hurt You?- Daniel Caesar is now playing
```

5. Add Song

```
Enter song title you want to add
Are You Okay?- Daniel Caesar
Are You Okay?- Daniel Caesar has been added
```

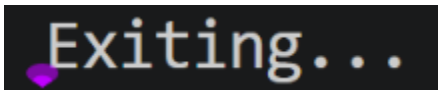
6. Remove Song

```
Enter song title to remove: Do You Like Me?- Daniel Caesar
Do You Like Me?- Daniel Caesar has been removed
```

**7. Deleting Playlist**




**8. Exit**




**Conclusion**

This activity helped me understand how doubly linked lists is implemented by using both next and prev pointers. I also gained experience implementing a circular linked list through a song playlist. The procedure started with list creation and traversal before moving on to insertion and deletion, which made it easier to understand how pointers are updated after each operation. The supplementary activity implements a music playlist using a circular linked list. Overall, I believe performed well in this activity because I was able to complete the required linked list operations. However, I still need to improve my understanding of circular linked lists and how it differs from doubly linked lists.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Grammarly**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Grammar correction**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **I used grammarly to improve and fix my grammar**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.